

ONLINE JUDGE SYSTEM

在线评测系统 实验报告

课程大作业 2 · FastAPI + Async SQLAlchemy + Streamlit

姓名	
学号	
班级	
仓库	<code>github.com/OverMoon64/online-judge-system</code>

可运行 · 可测试 · 可复现 · 面向课程本地验收

摘要

本项目实现了一个面向课程本地验收的在线评测系统。后端采用 FastAPI、SQLAlchemy Async 与 SQLite，前端采用 Streamlit；系统提供用户与角色管理、题目管理、动态语言配置、异步代码评测、提交检索与重测、公开日志与访问审计，以及基于 OpenAI-compatible Chat Completions 协议的可取消 AI 智能命题流程。

所有后端路由均为异步接口，响应统一为 {code, msg, data}，并返回真实 HTTP 状态码。判题器在独立临时目录中以参数数组启动编译器或解释器，同时限制时间、进程树内存与输出大小，支持 AC、WA、TLE、MLE、RE、CE、UNK。

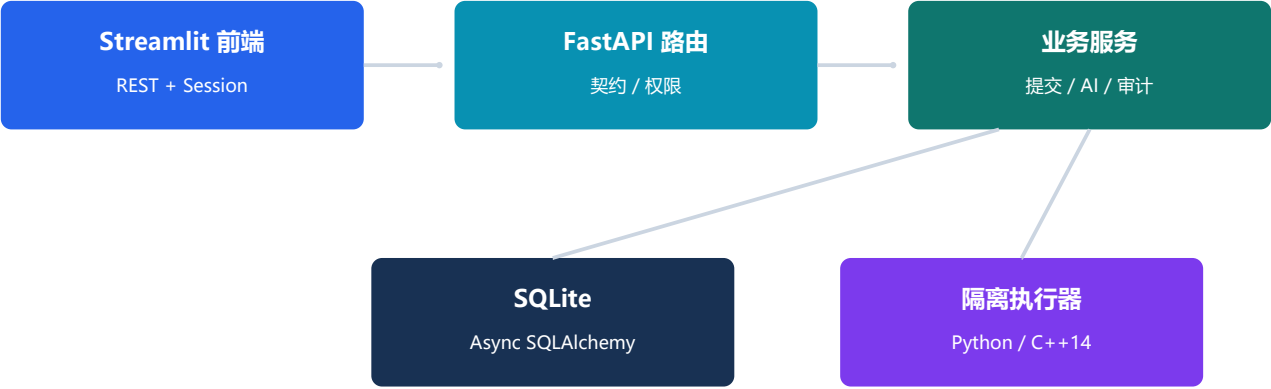
设计目标

目标	落实方式
契约准确	统一响应、422 转 400、固定错误优先级
异步完整	异步路由、数据库、子进程、HTTP 与后台任务
权限可证	Session、三角色、日志可见性与访问审计
执行可控	进程组、硬超时、RSS 监控、输出限制与清理
AI 可审阅	阶段进度、取消、费用、结构校验和参考解法试跑

说明：本系统定位为课程本地验收项目，不把进程级限制宣称为可直接暴露公网的强隔离沙箱。

1 系统架构

系统采用浏览器、REST API、业务服务、异步数据库与隔离执行器五层结构。Streamlit 不直接读取 SQLite，所有权限与数据约束均由 FastAPI 后端统一裁决。



分层职责

层	职责	关键技术
界面	账户、题目、提交、日志、AI 页面	Streamlit + HTTP Session
接口	校验、认证、错误映射、分页	FastAPI + Pydantic
业务	判题编排、统计、审计、AI 任务	asyncio + httpx
数据	事务、关系约束、初始化与重置	SQLAlchemy Async + SQLite
执行	编译运行、资源监控、清理	async subprocess + psutil

工程目录

app/ 后端核心 · frontend/ Streamlit · tests/ 自动化测试 · scripts/ 演示与报告 · .vscode/ WSL 开发配置

2 API 契约与权限

成功和失败响应均使用 {code, msg, data}。FastAPI 默认校验错误被转换为 400；组合失败条件严格执行 401 > 403 > 400 > 429 > 409 > 404 > 500，保证边界请求也具有确定行为。

认证与角色

角色	可执行操作	限制
user	浏览题目、提交代码、查看本人记录	受日志可见性和限流约束
admin	题目/语言/角色管理、全量提交、审计	仍经过统一数据校验
banned	无	旧会话也在依赖层被拒绝

会话与密码

Cookie 使用服务端密钥签名，仅保存用户标识；密码使用 bcrypt 哈希。启动和重置均确保存在 admin / admintestpassword。Session 中的角色不会被信任，每次请求重新读取数据库，因此角色变更和封禁立即生效。

日志访问矩阵

场景	结果	是否审计
管理员访问存在日志	允许	是
本人 + 题目公开日志	允许	是
本人 + 题目隐藏日志	允许	是
访问他人提交	403	是（资源存在）
未登录 / 参数错误 / 资源不存在	401 / 400 / 404	否

3 异步判题器

提交保存为 pending 后立即响应，后台任务再负责编译和逐点运行。该设计把 HTTP 延迟与用户程序耗时解耦，也使前端可以安全轮询进度。



后台任务立即接管；超时、超内存或取消时终止整个进程组并清理临时目录

安全执行链

措施	实现细节	防护目标
命令白名单	校验可执行程序、占位符与参数；不用 shell=True	命令注入
独立目录	每次评测使用临时工作目录	文件冲突与残留
进程组	新会话启动，结束时杀死完整进程树	孤儿进程
三类限制	硬超时、递归 RSS、输出上限	TLE/MLE/输出洪泛
最终清理	等待回收并删除临时目录	资源泄漏

结果判定

编译失败为 CE；运行异常为 RE；超过时间/内存分别为 TLE/MLE；退出正常后进行文本比较得到 AC/WA；无法分类的内部异常为 UNK。每个 AC 测试点 10 分。

输出比较

逐行移除行末空格并忽略最终多余换行，但保留前导空格。该规则容忍常见排版差异，同时避免把有意义的缩进误判为相同。

4 提交、统计与生命周期

提交列表支持用户、题目、状态筛选和分页。普通用户只能看到本人数据，管理员可以查看全部。限流按用户滚动一分钟计数，每分钟最多 3 次，并依照课程要求先于资源不存在检查。

重新评测

管理员对同一提交 ID 发起重测，系统清除旧结果并重新进入后台队列，而不是复制一条新提交。这保持了资源标识稳定，也便于验收时确认“同 ID 重测”。

一致性设计

需求	实现
提交总数	按用户计数全部提交
通过题数	仅统计 AC，并按 problem_id 去重
删除题目	事务内维护关联数据一致性
系统重置	先取消判题/AI 任务，再重建数据与管理员
退出会话	重置响应同时清除当前 Session

状态与故障恢复

业务状态写入数据库；内存中的后台任务注册表只负责运行期取消。服务重启不会暴露密钥，也不会把未完成任务误标为成功。内部异常通过统一处理器返回 500，避免泄露堆栈。

5 AI 智能命题

模型配置采用 OpenAI-compatible Chat Completions 协议，包含 URL、模型名、API Key、输入/输出单价、计价单位和币种。密钥只存于服务端内存，查询接口仅返回 has_api_key。

四阶段 workflow

阶段	进度	产物 / 校验
需求分析	10%	难度、知识点、约束与歧义分析
题面与解法	35%	题目结构、参考代码和初始样例
边界测试点	65%	补充极值、空值、重复值等边界
结构校验与试运行	85-100%	Pydantic 校验 + 参考解法全点 AC

Token 与费用

每次调用聚合输入、输出和总 Token，并按配置单价与计价单位计算费用；提供商未返回 usage 时，任务明确标记无法精确统计，绝不伪造数字。推荐验收时使用百炼 qwen3.7-plus，但代码不绑定厂商或模型。

可靠性与取消

模型输出先抽取 JSON，再经过严格结构校验和受限判题器试跑。首次失败会携带错误摘要发起一次修复请求。仍失败则进入 failed。取消 asyncio 任务会传播到正在进行的 HTTP 请求与本地试运行。

人工闸门：生成完成后只能一键载入题目编辑草稿；管理员检查题意、参考解法和测试点后才保存，避免未经审阅的模型输出直接进入题库。

6 Streamlit 前端

前端围绕真实实验收流程组织：账户、题目、提交、AI 命题和系统管理。独立 HTTP Session 保存签名 Cookie，所有操作均调用 REST API；按钮显隐只改善体验，权限最终仍由后端判断。

页面覆盖

页面组	核心交互
账户	注册、登录、退出、资料、用户列表与角色管理
题目	列表、详情、新增、编辑、删除、日志可见性
提交	代码编辑、语言选择、筛选分页、详情、日志、重测
AI 命题	模型配置、任务进度、取消、费用、载入编辑器
系统	语言配置、访问审计、数据重置

异步反馈

提交详情使用局部刷新显示 pending/running 到最终状态，不阻塞其它页面；所有 API 错误统一展示 HTTP 状态、code 与 msg。AI 页面同样展示阶段进度、取消状态和费用统计。

真实界面截图



图：Streamlit 在线评测系统主界面（本地运行）

7 测试与质量保障

测试分为纯接口测试、Linux 真实判题集成测试、AI 假提供商测试和人工浏览器流程。测试数据库逐用例隔离，避免状态相互污染。

自动化结果

Ubuntu 22.04 · Python 3.10.12 · G++ 11.4.0: 23 个测试全部通过。总行覆盖率 92.78%，超过 CI 的 85% 门槛。Ruff 静态检查与格式检查均通过。

测试域	代表场景
契约	字段校验、统一响应、422→400、错误优先级
认证	注册、登录、Session、角色变化、封禁
判题	Python/C++: AC、WA、RE、CE、TLE、MLE、空白比较
数据	分页筛选、限流、去重统计、同 ID 重测、重置
审计	公开/隐藏日志与成功/拒绝访问记录矩阵
AI	合法/非法 JSON、usage/费用、取消、结构与试运行

持续集成

GitHub Actions 在 Ubuntu 和 Python 3.10 上安装 G++，运行 Ruff 静态检查、格式检查与 pytest，并设置 85% 覆盖率门槛。VS Code 配置对应的 WSL 解释器、后端/前端任务、调试入口和测试入口。

8 安全边界与改进方向

当前实现通过命令白名单、无 Shell 执行、临时目录、进程组和资源限制降低本地评测风险，足以服务课程验收；但进程级措施并不是公网多租户环境所需的强隔离。

若部署到公网

- 将评测迁移到一次性容器或专用沙箱；使用非特权用户、只读根文件系统和网络命名空间。
- 使用 cgroup v2、seccomp 和严格挂载策略，把 CPU、内存、进程数与系统调用限制交给内核。
- 引入消息队列和独立 Judge Worker，增加幂等任务、重试、崩溃恢复和横向扩展。
- 把 SQLite 替换为生产数据库，将审计日志持久化，并配置 HTTPS、CSRF 与速率限制网关。

隐私与密钥

API Key 不写数据库、日志、Git 或响应；.env、数据库、临时文件、缓存与报告构建产物均按用途纳入 .gitignore。示例配置不包含真实秘密。

AI 使用说明

本项目使用 AI 辅助需求梳理、代码实现、测试设计与报告排版。所有内容均通过静态检查、自动化测试或人工审阅验证；验收中的 AI 生成题目也必须经管理员审核。

9 结论

本实验把异步 Web API、数据库、操作系统进程管理、权限审计和大模型工作流整合为一个完整系统。实现重点不仅是运行代码，更是在明确 API 契约下正确处理并发、资源回收、权限优先级、数据一致性与失败路径。

项目提供 VS Code Remote WSL 配置、演示种子数据、验收清单、自动化测试和 CI，使开发、验证与演示均可重复执行。AI 命题通过结构验证、参考解法试运行和人工闸门，在可用性与可靠性之间取得平衡。

交付物

交付	位置
源代码	GitHub: OverMoon64/online-judge-system
运行指南	README.md
验收流程	docs/demo-checklist.md
报告源稿	docs/report.md
PDF 报告	output/pdf/online-judge-system-report.pdf

END OF REPORT